DD: BIL-04 — Dunning Engine

Developer implementation guide: `DD_BIL-04-Dunning_Engine-DEV_IMPL_GUIDE-v1.4.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_BIL-04-Dunning_Engine-v1.4.md`.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue **Orchestration**: Synchronous service classes per `SUB-WF-FRAMEWORK-01` (Java/Spring OR Laravel/PHP per-module choice). Scheduled scanner jobs for the recurring dunning evaluation loop.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-04-Dunning_Engine-v1.4.md`
Version	`v1.4`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `GET /api/dunning-programs`
- `GET /api/dunning-programs/{code}?version={v}`
- `GET /api/dunning-states/pending-termination-review`
- `GET /api/dunning-states/{subscriptionId}`
- `GET /api/dunning-states/{subscriptionId}/history`
- `GET /api/dunning-states?level={n}&status={s}&limit=50`
- `POST /api/dunning-programs`
- `POST /api/dunning-programs/{code}/new-version`
- `POST /api/dunning-states/refresh-debt`

- `POST /api/dunning-states/{id}/admin-clear`
- `POST /api/dunning-states/{id}/admin-clear-restrictions`
- `POST /api/dunning-states/{id}/advance`
- `POST /api/dunning-states/{id}/clear-without-payment`
- `POST /api/dunning-states/{id}/extend-review`
- `POST /api/dunning-states/{id}/force-terminate`
- `POST /api/dunning-states/{id}/hold`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `advance-cycle`
- `billing-dunning`
- `billing-rules`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `InvoiceOverdue`
- `PaymentApplied`
- `SubscriptionDunningCleared`
- `SubscriptionDunningDebtIncreased`
- `SubscriptionDunningRecoveryFailed`
- `SubscriptionDunningTerminationPending`
- `SubscriptionEnteredDunning`
- `SubscriptionPaused`
- `SubscriptionRestrictionAdded`
- `SubscriptionResumed`
- `SubscriptionSuspendedForNonPayment`
- `SubscriptionTerminated`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.

- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue

Orchestration: Synchronous service classes per `SUB-WF-FRAMEWORK-01` (Java/Spring OR Laravel/PHP per-module choice). Scheduled scanner jobs for the recurring dunning evaluation loop.

A — Target

1. What this process is about

BIL-04 is the **dunning engine** — the module that detects when a Subscription has unpaid debt past its grace period and drives a configurable escalation policy: restrict → suspend → terminate. It is the bridge between BIL-01 / BIL-02 (which know what's owed) and SUB-WF-SUSPEND-NP-01 / SUB-WF-TERMINATE-01 / SUB-WF-RESTRICT-01 (which apply the consequences).

The engine runs continuously: scheduled scanners periodically evaluate every Subscription against operator-configured dunning policy, advance Subscriptions through dunning levels at the configured cadence, and fire the appropriate downstream workflow (restriction, suspension, termination) at each escalation step. When a customer pays, BIL-04 also drives the **resume path** — calling SUB-WF-RESUME-01 with the `DUNNING_RESUME` trigger or removing dunning-applied restrictions via SUB-WF-RESTRICT-01.

The engine is fundamentally a **state machine per Subscription**, with the state stored in a dedicated `dunning_state` row. Each Subscription with unpaid debt has exactly one dunning state row tracking: which dunning level it's at, when it entered,

what restrictions are applied, what the outstanding debt is. The scheduled scanner advances the state machine; payment events received from BIL-01 retreat or clear it.

Out of scope:

- **Detecting overdue invoices** — owned by BIL-02 (invoicing). BIL-02 emits `InvoiceOverdue` events for postpaid Subscriptions; BIL-04 consumes them.
- **Detecting insufficient wallet at cycle boundary for PREPAID subscriptions** — owned by BIL-03 (cycle close). At cycle boundary BIL-03 queries BIL-05 wallet balance; if insufficient, fires `CyclePaymentMissed` (same event used by PREPAYMENT — uniform dunning trigger). BIL-04 consumes and enters the Subscription into dunning. PREPAID is wallet-based (customer tops up anytime, cycle debits at boundary) and is distinct from PREPAYMENT (per-cycle invoice paid before cycle starts).
- **Detecting unpaid upcoming cycles in pre-payment subscriptions** — owned by BIL-03 (cycle close). For pre-payment subscriptions (Yas Senegal), the customer must pay before each cycle starts. At cycle boundary, BIL-03 checks whether the upcoming cycle has been paid; if not, it emits `CyclePaymentMissed` and BIL-04 enters the Subscription into dunning.
- **Computing what's owed** — BIL-04 reads outstanding-debt totals from BIL-01 / BIL-02 via REST. Does not own the money math.
- **Actually charging restriction-lift fees or reactivation fees** — owned by BIL-01 (via the standard charge endpoint with the appropriate intent code). BIL-04 just triggers the workflows that drive the billing.
- **Settling outstanding debt at termination** — owned by BIL-01 + the termination workflow. BIL-04 just decides when to trigger TERMINATE; the termination workflow's billing call handles write-off.
- **The actual restriction / suspension / termination state mutation on the Subscription** — owned by SUB-WF-RESTRICT-01 / SUB-WF-SUSPEND-NP-01 / SUB-WF-TERMINATE-01. BIL-04 just calls them with the right context.
- **Notification dispatch (payment reminders, dunning notices)** — owned by the `notification` module (DD_NOT-01, planned). BIL-04 emits Kafka events at each dunning level transition; notification consumes.

2. Business rules

Rules grouped by scope: `D` for dunning policy and state machine, `E` for evaluation cadence, `T` for trigger conditions, `R` for resume / recovery, `C` for catalog discipline, `WF` for workflow integration.

Dunning policy and state machine rules

ID	Rule	Triggered on
R-BIL-04-D-1	Each Subscription with unpaid debt is in exactly one **dunning state**, materialized as a row in `dunning_state` keyed by `subscription_id`. The state row carries: current `dunning_level` (`0` = none / cleared, `1` = first-warning, `2` = restricted, `3` = suspended, `4` = terminated), `entered_level_at`, `outstanding_debt_amount`, `outstanding_debt_currency`, `dunning_program_ref` (which policy applies), `last_evaluated_at`, `next_evaluation_at`.	Dunning enter / advance / recover
R-BIL-04-D-2	The dunning state machine is **monotonic forward** during dunning escalation: a Subscription advances from level N to level N+1 only after the configured `grace_period_at_level_N_days` has elapsed since `entered_level_at`. Skipping levels is NOT allowed (no jumping from level 1 to level 3 without passing through 2).	Scanner evaluation
R-BIL-04-D-3	The dunning state machine **recovers backward to level 0** when debt is fully cleared. Partial payment that reduces but does not eliminate the debt does NOT change the dunning level — the Subscription stays at its current level with reduced `outstanding_debt_amount`. The escalation clock (`entered_level_at`) is preserved. Full payment clears the dunning state and triggers the appropriate recovery action (lift restriction, resume from suspension, etc.).	Payment received

ID	Rule	Triggered on
R-BIL-04-D-4	The dunning state row's lifecycle: created when a Subscription first enters dunning (BIL-02 emits `InvoiceOverdue` for postpaid, or BIL-03 emits `CyclePaymentMissed` for pre-payment), updated at each evaluation, **soft-deleted** (kept for audit with `status = CLEARED`) when debt clears, **archived** (moved to `dunning_state_archive` table) when the Subscription terminates and the dunning state has been settled. Hard delete is never performed in V3 — the row is part of the regulatory audit trail.	State transitions
R-BIL-04-D-5	Each `dunning_state` row references a `dunning_program` config row that defines: which levels exist, what the grace period is at each level, what action each level applies (RESTRICT_ADD with which restriction code / SUSPEND_NP / TERMINATE), and any per-level fee. Different `dunning_program` rows enable per-operator scoping. In V3 three billing models are supported: `POSTPAID` (some Wananchi customers — bill after consumption, pay by invoice due date), `PREPAID` (majority of Wananchi customers — wallet-based, customer tops up wallet, cycle charge debited at cycle boundary), and `PREPAYMENT` (Yas Senegal — pay each specific cycle invoice before the cycle starts, no wallet). The `billing_mode` column on the program stores the model. Note that PREPAID and PREPAYMENT are architecturally distinct despite being similar customer experiences: PREPAID has a persistent wallet balance and topup-anytime; PREPAYMENT has no wallet, just per-cycle invoices paid before each cycle.	Dunning state lookup

Evaluation cadence rules

ID	Rule	Triggered on
R-BIL-04-E-1	A **dunning evaluation scanner** runs every 15 minutes, scanning `dunning_state` rows where `next_evaluation_at <= now()` AND `status = ACTIVE`. For each due row, it: (1) refreshes `outstanding_debt_amount` from BIL-01 / BIL-02, (2) decides whether to advance, hold, or recover, (3) executes the decision by invoking the appropriate workflow, (4) updates `next_evaluation_at` to the next scheduled time per the dunning program.	Scheduler
R-BIL-04-E-2	The `next_evaluation_at` advance is **at-most-daily** per Subscription (cap the scanner's ambition — even if multiple events fire on the same Subscription on the same day, evaluation happens once per cadence boundary). The boundary is configurable per dunning program; default 24 hours.	Scanner
R-BIL-04-E-3	Per-Subscription serialization: a `SELECT FOR UPDATE` of the `dunning_state` row at evaluation start ensures only one node processes a given Subscription at a time.	Scanner
R-BIL-04-E-4	Batch size: each scanner run processes at most **500 dunning states per cycle** to prevent monopolizing the workflow modules. Subscriptions not processed in a given run are evaluated in subsequent runs based on `next_evaluation_at` ordering.	Scanner
R-BIL-04-E-5	A **payment-arrival event listener** processes `PaymentApplied` events from BIL-01 in near-real-time (Kafka consumer, no scanner delay). For each event: refreshes outstanding debt for the affected Subscription, and if debt is now zero, triggers the recovery path immediately — does not wait for the next scanner run.	Kafka consumer

Trigger condition rules

ID	Rule	Triggered on
R-BIL-04-T-1	A Subscription **enters dunning at level 1** when BIL-04 receives one of the trigger events: `InvoiceOverdue` from BIL-02 (postpaid model, some Wananchi customers — invoice past due date with no payment) OR `CyclePaymentMissed` from BIL-03 (used for both PREPAID wallet-insufficient-at-cycle AND PREPAYMENT cycle-invoice-unpaid scenarios — the event payload's `billingMode` field distinguishes them). The entry creates a `dunning_state` row with `dunning_level = 1`, `entered_level_at = now()`, `entered_dunning_at = now()`, `dunning_program_ref` resolved by lookup (per operator + billing mode), `triggering_event_type` recording which event fired, `triggering_event_ref` capturing the source identifier (invoice ID, cycle reference, or wallet snapshot ID), and emits `SubscriptionEnteredDunning` Kafka event.	Entry event received
R-BIL-04-T-2	A Subscription is **already in dunning at level N**, and a new overdue event arrives: BIL-04 refreshes the `outstanding_debt_amount` and emits `SubscriptionDunningDebtIncreased` if the new amount is higher. It does NOT advance the dunning level or reset the entered-level clock — additional overdue invoices accumulate into the same dunning episode. The clock advances only via the scanner per the grace-period schedule.	Additional event received
R-BIL-04-T-3	Advance from level N to level N+1 requires: (a) the configured `grace_period_at_level_N_days` has elapsed since `entered_level_at`, AND (b) outstanding debt is still > 0 at evaluation time. If debt has cleared in the interim (e.g., partial payment + further charges netting to zero), recovery is triggered instead.	Scanner advance step
R-BIL-04-T-4	Advance to level 4 (TERMINATE) requires an additional safeguard: a configurable `pre_termination_review_required` flag on the dunning program. If TRUE, the scanner does NOT auto-trigger TERMINATE; instead it transitions the dunning state to `PENDING_TERMINATION_REVIEW`, emits `SubscriptionDunningTerminationPending` Kafka event, and waits for an operator to either confirm (via admin endpoint) or extend (push next-evaluation-at out). Default: TRUE for all V3 programs (postpaid Wananchi and pre-payment Yas SN both deserve manual review before permanent termination — operator preference).	Scanner pre-TERMINATE
R-BIL-04-T-5	A Subscription's dunning state does NOT advance if the Subscription is in any of these statuses: `CREATED` (not yet activated, no dunning applicable), `TERMINATED` (already gone), `RETIRED` (archival). For `PENDING_*` states (mid-workflow), the scanner skips and re-evaluates on the next cycle. For `SUSPENDED` from voluntary pause (not from dunning), the dunning state is paused alongside the Subscription — see R-BIL-04-T-6.	Scanner
R-BIL-04-T-6	When a Subscription is voluntarily paused (SUB-WF-PAUSE-01), any active dunning state is set to `status = SUSPENDED_BY_PAUSE`, with `next_evaluation_at = null`. When the Subscription resumes from voluntary pause, the dunning state reverts to `status = ACTIVE` and `next_evaluation_at` is set to `now() + 1 day` (immediate re-evaluation on the next scanner cycle). This prevents dunning from continuing to escalate against a customer who has correctly paused their service for an approved reason.	Voluntary pause / resume

Resume / recovery rules

ID	Rule	Triggered on
R-BIL-04-R-1	When `outstanding_debt_amount` reaches 0 (full payment received), the dunning state's recovery action depends on the current `dunning_level`: level 1 (warning only) → mark CLEARED, emit `SubscriptionDunningCleared`; level 2 (restricted) → call SUB-WF-RESTRICT-01 to remove dunning-applied restrictions, then mark CLEARED; level 3 (suspended) → call SUB-WF-RESUME-01 with `DUNNING_RESUME` trigger; level 4 (terminated) → no recovery, the Subscription is gone.	Debt cleared event
R-BIL-04-R-2	Recovery from level 2 (restricted): BIL-04 calls SUB-WF-RESTRICT-01 with intent `REMOVE` for each restriction code listed in the dunning state's `applied_restriction_codes` array, in sequence. Each removal is its own workflow call with its own idempotency key. On any restriction-removal failure, BIL-04 stops, sets the dunning state to `status = RECOVERY_FAILED`, emits `SubscriptionDunningRecoveryFailed`, and surfaces for operator review.	Recovery step
R-BIL-04-R-3	Recovery from level 3 (suspended): BIL-04 calls SUB-WF-RESUME-01 with `DUNNING_RESUME` trigger. The resume workflow re-enables service. On successful resume, BIL-04 marks the dunning state CLEARED. On resume failure, dunning state stays SUSPENDED and BIL-04 retries on next scanner cycle up to 3 times before flagging RECOVERY_FAILED.	Recovery step
R-BIL-04-R-4	Restrictions applied by BIL-04 carry the marker `systemManaged = true` and the dunning state row holds the list of applied restriction codes. The corresponding rule R-SUB-WF-RESTRICT-01-C-4 prevents human REMOVE on these — operators wanting to lift them must instead clear the debt or use an admin override endpoint exposed by BIL-04 (`POST /api/dunning-states/{id}/admin-clear` with audit).	Restriction application + removal
R-BIL-04-R-5	An admin may **override dunning escalation** at any time for exceptional cases: (a) hold (postpone next evaluation), (b) skip-to-next-level (force advance ahead of schedule), (c) clear-without-payment (write-off debt, recover the Subscription). All overrides are role-gated to `DUNNING_ADMIN` role and audited.	Admin actions
R-BIL-04-R-6	For pre-payment subscriptions (`PREPAYMENT` billing mode, Yas SN): when payment is received during dunning, the payment activates a **fresh cycle starting from the payment date**. The previously missed cycle is NOT retroactively activated — the customer does not get back-billed for the suspended period, and does not receive service for that period. This is the standard pre-payment recovery model and is the responsibility of BIL-01 (which knows how to align cycle anchors on payment). BIL-04 receives `PaymentApplied` and triggers recovery as usual; BIL-01 has already aligned the cycle. The Subscription's `current_cycle_start` is reset by BIL-01 as part of the payment processing — BIL-04 does not coordinate this.	Pre-payment recovery
R-BIL-04-R-7	For postpaid subscriptions (`POSTPAID` billing mode, some Wananchi customers): when payment is received during dunning, the payment settles the overdue debt and the Subscription continues on its existing cycle anchor. No cycle realignment. Recovery follows the standard R-1..R-3 paths.	Postpaid recovery

ID	Rule	Triggered on
R-BIL-04-R-8	For prepaid wallet subscriptions (`PREPAID` billing mode, majority of Wananchi customers): BIL-04 listens for `WalletToppedUp` events from BIL-05 (Wallet & Topup module). When a topup arrives for a Subscription currently in dunning, BIL-04 queries BIL-05 for the new wallet balance and queries the Subscription's current cycle charge requirement. If the new balance is sufficient to cover the cycle: BIL-04 triggers recovery (debits the wallet via BIL-05 for the cycle charge, advances cycle anchor via BIL-03 internal endpoint, removes restrictions / resumes service as appropriate per the standard R-1..R-3 paths, emits `SubscriptionDunningCleared`). If the topup leaves the balance still insufficient: BIL-04 emits `SubscriptionDunningDebtIncreased` with a corrected outstanding-amount calculation (cycle charge minus wallet balance shows remaining shortfall) and stays at the current dunning level. The recovery is **automatic** — no operator action needed. This is BIL-04's PREPAID-specific recovery path and is symmetric to R-7 (postpaid debt cleared via payment) but uses the wallet rather than payment-against-invoice.	PREPAID wallet recovery

Catalog discipline rules

ID	Rule	Triggered on
R-BIL-04-C-1	`dunning_program` is a versioned catalog (akin to BIL-CFG-01 versioning): edits create new versions, never mutate published ones. The `dunning_state` row pins its program version at creation time — in-flight dunning episodes continue under the policy that was in effect when they started, regardless of subsequent program edits.	Program version pinning
R-BIL-04-C-2	A `dunning_program` declares its levels (contiguous, 1 to N, N typically 3-4), each level's `grace_period_days`, each level's `action_workflow_intent` (`WARNING_ONLY`, `RESTRICTION_ADD`, `SUSPEND_NP`, `TERMINATION`), each level's `action_payload` shape per the schema in §Data model, and optional customer-portal message codes and visibility windows. Fees applicable at any escalation step (e.g., reactivation fee on dunning resume) are NOT declared in the program — they are owned by BIL-CFG-01 BillableEvents that match on the workflow intent code (`RESTRICTION_REMOVE_INTENT`, `RESUME_NP_INTENT`, etc.) at billing-call time. The dunning program declares only the orchestration, not the money.	Program definition
R-BIL-04-C-3	Dunning programs are scoped: each row has `operator_code` (`WANANCHI_KE` / `WANANCHI_UG` / `WANANCHI_TZ` / `YAS_SN` / `*` for default) and `billing_mode` (`POSTPAID` for Wananchi postpaid customers, `PREPAID` for Wananchi wallet-based customers, `PREPAYMENT` for Yas SN). Resolution at Subscription entry uses operator + billing mode + most-specific match. Wananchi operates programs for both POSTPAID and PREPAID concurrently — each Subscription's billing mode (set at activation per SUB-LM-01) selects which program applies.	Program lookup at entry
R-BIL-04-C-4	A program may declare `pre_termination_review_required: true` (default for postpaid) — when set, level 4 (TERMINATE) does not auto-advance; it requires operator confirmation as documented in R-BIL-04-T-4.	Program field

Workflow integration rules

ID	Rule	Triggered on
R-BIL-04-WF-1	BIL-04 calls SUB-WF-RESTRICT-01, SUB-WF-SUSPEND-NP-01, SUB-WF-RESUME-01 (with `DUNNING_RESUME`), and SUB-WF-TERMINATE-01 (with `DUNNING_TERMINATION` trigger) via REST. Each call uses an idempotency key of the form `dunning-{subscription_id}-{action_type}-{dunning_program_version}-{entered_level_at}`. This makes BIL-04's retries safe (downstream dedups).	Workflow invocation
R-BIL-04-WF-2	The actor on these calls is `service:bil-04` (a service-account role per FOUNDATION_AUTH), and the channel is `SYSTEM`. The workflows recognize these and apply their dunning-specific role-gating rules (e.g., R-SUB-WF-SUSPEND-NP-01-T-1 only allows `BILLING_INTERNAL` to trigger SUSPEND-NP).	Workflow invocation
R-BIL-04-WF-3	If a workflow call returns a synchronous failure (e.g., FUL-04 timeout during SUSPEND-NP), BIL-04 logs the failure on the `dunning_state` row (`last_workflow_failure_code`, `last_workflow_failure_at`), retries on the next scanner cycle with backoff (initially 1 hour, then 4 hours, then 24 hours), and after 3 failed attempts marks the dunning state `RECOVERY_FAILED` for operator review.	Workflow failure
R-BIL-04-WF-4	BIL-04 listens for the workflows' Kafka completion events (`SubscriptionRestrictionAdded`, `SubscriptionSuspendedForNonPayment`, `SubscriptionResumed`, `SubscriptionTerminated`) to confirm its workflow calls actually committed. If a workflow's synchronous response said success but the corresponding Kafka event does not arrive within 5 minutes, BIL-04 flags an inconsistency for operator review (the workflow may have committed but Kafka outbox is lagging — or there's a real consistency bug).	Workflow confirmation

3. Module owner and impacted modules

Role	Module	What it does in this process
Owner	New `billing-dunning` module	Owns the dunning state machine, the dunning program catalog, the scheduled scanner, the Kafka consumers for overdue/payment events, the REST admin endpoints, the Kafka event publication.
Impacted (upstream events consumed)	BIL-02 (`InvoiceOverdue` for postpaid), BIL-03 (`CyclePaymentMissed` for both PREPAID wallet-insufficient and PREPAYMENT cycle-unpaid), BIL-01 (`PaymentApplied`, `ChargeApplied`), BIL-05 (`WalletToppedUp` for PREPAID recovery), SUB-LM-01 (`SubscriptionPaused`, `SubscriptionResumed` from voluntary pause/resume — for the SUSPENDED_BY_PAUSE handling)	Source events drive dunning state transitions.
Impacted (callees via REST)	SUB-WF-RESTRICT-01 (add + remove), SUB-WF-SUSPEND-NP-01, SUB-WF-RESUME-01 (DUNNING_RESUME trigger), SUB-WF-TERMINATE-01 (DUNNING_TERMINATION trigger), BIL-01 (debt query), BIL-05 (wallet balance query and wallet debit for PREPAID recovery), BIL-03 (cycle anchor advance via internal endpoint for PREPAID auto-recovery)	Receive synchronous REST calls from BIL-04's scanner / recovery handlers.
Impacted (downstream consumers via Kafka)	`notification` module (DD_NOT-01, planned), CVM, analytics, customer portal (visible dunning state)	Consume BIL-04's Kafka events.
Frontend	Admin dunning console (operator view: in-flight dunning states, drill-in per Subscription, override actions); customer portal (read-only view of "your service may be suspended on date X if not paid by Y")	Read from BIL-04's REST endpoints.

Role	Module	What it does in this process
Engines	Drools (for evaluating dunning rules with operator-customizable conditions if needed in V3.2) — V3 keeps rules in code	Validation.

4. Foundation references

DB → `FOUNDATION_DB.md` · Drools → `FOUNDATION_DROOLS.md` · Kafka → `FOUNDATION_KAFKA.md` · Auth → `FOUNDATION_AUTH.md`. Framework: SUB-WF-FRAMEWORK-01 (BIL-04 follows its synchronous-service-class pattern for outbound workflow invocations, and uses the scheduled-scanner pattern for the recurring evaluation loop).

Technical design

Module placement

Layer	Where
Frontend	New admin dunning console (operator view of dunning states, manual override actions), customer portal (read-only dunning notice + payment link)
Backend	New `billing-dunning` module on cluster C (revenue cluster, alongside `billing` and `subscription`).
Rules	DRL contributions to `billing-rules` KJAR under `rules/billing-dunning/`. In V3, most dunning logic is in Java/PHP code; DRL is reserved for V3.2 operator-customizable conditions if needed.
Events consumed	`sophix.billing.invoice.overdue` (postpaid), `sophix.billing.cycle.paymentmissed` (PREPAID + PREPAYMENT), `sophix.billing.payment.applied`, `sophix.billing.wallet.toppedup` (PREPAID recovery), `sophix.subscription.subscription.paused`, `sophix.subscription.subscription.resumed`. Also confirmation events: `sophix.subscription.restrictionadded`, `sophix.subscription.subscription.suspendedfornonpayment`, `sophix.subscription.subscription.resumed`, `sophix.subscription.subscription.terminated`.
Events emitted	`sophix.billing.dunning.entered`, `sophix.billing.dunning.advanced`, `sophix.billing.dunning.debtincreased`, `sophix.billing.dunning.cleared`, `sophix.billing.dunning.recoveryfailed`, `sophix.billing.dunning.terminationpending`, `sophix.billing.dunning.adminoverride`.

Dunning state machine

[Embedded image omitted: Dunning state machine]

Dunning evaluation flow per Subscription

[Embedded image omitted: Dunning evaluation flow]

Codebase

Java/Spring Boot layout

```
modules/billing-dunning/
  src/main/java/com/sophix/billing/dunning/
    web/rest/
      DunningStateResource.java                ← admin read + override endpoints
      DunningProgramResource.java              ← catalog admin
      dto/...
    service/
      DunningEvaluationService.java            ← core evaluation logic per Subscription
      DunningEntryService.java                 ← creates dunning_state on entry events
      DunningRecoveryService.java              ← drives recovery on payment events
      DunningProgramResolver.java              ← tenant + billing-mode → program version
      DunningWorkflowInvoker.java              ← calls SUB-WF-* with proper idempotency keys
    domain/
      DunningState.java                        ← entity
      DunningProgram.java                      ← versioned catalog entity
      DunningLevel.java                        ← enum: NONE | LEVEL_1_WARNING | LEVEL_2_RESTRICTED | LEVEL_3_SUSPENDED
```

```
    DunningStateStatus.java                    ← enum: ACTIVE | CLEARED | SUSPENDED_BY_PAUSE | PENDING_TERMINATION_REV
    DunningActionType.java                     ← enum: WARNING_ONLY | RESTRICTION_ADD | SUSPEND_NP | TERMINATE
  repository/
    DunningStateRepository.java
    DunningProgramRepository.java
    DunningStateArchiveRepository.java
  scanner/
    DunningEvaluationScanner.java              ← @Scheduled(fixedDelay = 900000) every 15 min
    DunningArchiveScanner.java                 ← nightly archive of CLEARED+TERMINATED rows older than retention thres
  kafka/
    InvoiceOverdueConsumer.java
    CyclePaymentMissedConsumer.java            ← PREPAID wallet-insufficient AND PREPAYMENT cycle-unpaid trigger
    WalletToppedUpConsumer.java                 ← PREPAID recovery trigger for Wananchi wallet customers
    PaymentAppliedConsumer.java
    SubscriptionPausedResumedConsumer.java      ← handles SUSPENDED_BY_PAUSE transition
    WorkflowConfirmationConsumer.java           ← confirms BIL-04's workflow calls via Kafka events
  event/
    DunningEnteredEventPublisher.java
    DunningAdvancedEventPublisher.java
    DunningDebtIncreasedEventPublisher.java
    DunningClearedEventPublisher.java
    DunningRecoveryFailedEventPublisher.java
    DunningTerminationPendingEventPublisher.java
    DunningAdminOverrideEventPublisher.java
  api/
    BillingDebtApi.java                        ← REST client to BIL-01 / BIL-02 for outstanding debt query
    WalletApi.java                             ← REST client to BIL-05 for wallet balance query + debit
    CycleAnchorApi.java                        ← REST client to BIL-03 for cycle anchor advance after PREPAID recovery
    SubWfRestrictApi.java                      ← REST client to SUB-WF-RESTRICT-01
    SubWfSuspendNpApi.java                     ← REST client to SUB-WF-SUSPEND-NP-01
    SubWfResumeApi.java                        ← REST client to SUB-WF-RESUME-01
    SubWfTerminateApi.java                     ← REST client to SUB-WF-TERMINATE-01
src/main/resources/
  config/liquibase/changelog/billing-dunning/
    001_dunning_program.xml
    002_dunning_state.xml
    003_dunning_state_archive.xml
  rules/billing-dunning/
    (reserved for V3.2 operator-customizable conditions; V3 logic in Java)
```

Laravel/PHP parallel layout

```
sophix/billing-dunning/
  src/Http/Controllers/
    DunningStateController.php
    DunningProgramController.php
  src/Http/Requests/
    DunningOverrideRequest.php
    DunningProgramRequest.php
  src/Service/
    DunningEvaluationService.php
    DunningEntryService.php
    DunningRecoveryService.php
    DunningProgramResolver.php
    DunningWorkflowInvoker.php
  src/Domain/
    DunningState.php
    DunningProgram.php
    DunningLevel.php
    DunningStateStatus.php
    DunningActionType.php
  src/Repository/
    DunningStateRepository.php
    DunningProgramRepository.php
    DunningStateArchiveRepository.php
  src/Console/
    EvaluateDunningStatesCommand.php           ← scheduled via app/Console/Kernel.php everyFifteenMinutes()
    ArchiveDunningStatesCommand.php            ← scheduled nightly
  src/Kafka/
    InvoiceOverdueListener.php
    CyclePaymentMissedListener.php             ← PREPAID wallet-insufficient AND PREPAYMENT cycle-unpaid trigger
    WalletToppedUpListener.php                 ← PREPAID recovery trigger for Wananchi wallet customers
    PaymentAppliedListener.php
    SubscriptionPausedResumedListener.php
    WorkflowConfirmationListener.php
  src/Event/
    DunningEnteredEventPublisher.php
    DunningAdvancedEventPublisher.php
    DunningDebtIncreasedEventPublisher.php
    DunningClearedEventPublisher.php
    DunningRecoveryFailedEventPublisher.php
    DunningTerminationPendingEventPublisher.php
    DunningAdminOverrideEventPublisher.php
```

```
src/Api/
  BillingDebtApi.php
  WalletApi.php                                ← PREPAID wallet balance + debit
  CycleAnchorApi.php                           ← BIL-03 cycle anchor advance after PREPAID recovery
  SubWfRestrictApi.php
  SubWfSuspendNpApi.php
  SubWfResumeApi.php
  SubWfTerminateApi.php
src/Providers/
  BillingDunningServiceProvider.php
routes/
  api.php
database/migrations/
  2026_07_01_create_dunning_program_table.php
  2026_07_01_create_dunning_state_table.php
  2026_07_01_create_dunning_state_archive_table.php
rules/billing-dunning/                         ← reserved for V3.2
```

The DRL package (when reserved rules are added in V3.2) is always Java/Maven regardless of which stack owns BIL-04; Laravel modules call KIE Server over REST per FOUNDATION_DROOLS.md.

Foundation-specific configurations

Foundation	Configuration
DB	New tables on cluster C (revenue): `dunning_program` (versioned catalog, ~20-50 rows), `dunning_state` (active dunning episodes, ~5-10k rows for Wananchi target), `dunning_state_archive` (terminated / cleared rows kept for audit, grows monotonically — partition monthly per FOUNDATION_DB.md audit pattern). `_audit` shadow tables on all three.
Drools	DRL package reserved at `rules/billing-dunning/` for V3.2. V3 logic in code.
Kafka	Consumes 6 topics, emits 7 topics (all on cluster C Kafka brokers per FOUNDATION_KAFKA.md).
Auth	Roles: `DUNNING_ADMIN` (manual overrides), `DUNNING_OPERATOR` (read-only investigation), `BILLING_INTERNAL` (BIL-04's outbound calls to subscription workflows — already a recognized role in the workflow DDs).

Frontend

Screen	Purpose
Admin Dunning Console — Dashboard	Counts per dunning level, recent transitions, recovery-failed alerts, pending-termination-review queue
Admin Dunning Console — Subscription Drill-in	One Subscription's dunning state history, current debt, applied restrictions, level timeline, next scheduled evaluation, override actions (hold, advance, clear-without-payment, force-terminate)
Admin Dunning Console — Pending Termination Review	Queue of dunning states at PENDING_TERMINATION_REVIEW; per-row confirm-terminate / extend / clear-without-payment actions
Admin Dunning Console — Program Catalog	View and edit `dunning_program` catalog (creating new versions; never editing published ones)
Customer Portal — Dunning Notice	Read-only banner shown when customer's Subscription is in dunning: current level, debt amount, next escalation date, "Pay now" CTA linking to BIL-02 invoice payment

Data model

`dunning_program` — versioned catalog

Column	Type	Notes
`id`	TEXT PK	ULID
`code`	TEXT NOT NULL	Stable identifier (e.g., `wananchi_ke_postpaid_standard`, `yas_sn_fiber_prepayment`)
`version`	INTEGER NOT NULL	Auto-increment per `code` on each edit
`description`	TEXT NOT NULL	
`operator_code`	TEXT NOT NULL	`WANANCHI_KE` / `WANANCHI_UG` / `WANANCHI_TZ` / `YAS_SN` / `*`

Column	Type	Notes
billing_mode	TEXT NOT NULL	POSTPAID (some Wananchi customers), PREPAID (majority of Wananchi customers — wallet-based), or PREPAYMENT (Yas SN — per-cycle invoice paid before cycle starts).
level_definitions	JSONB NOT NULL	Array of level configs — see below
pre_termination_review_required	BOOLEAN NOT NULL	Default TRUE for both POSTPAID and PREPAYMENT in V3 (manual review preferred before permanent termination for all V3 operators)
published_at	TIMESTAMPTZ NOT NULL	When this version became active
retired_at	TIMESTAMPTZ NULL	When superseded by a later version
created_by	TEXT NOT NULL	

UNIQUE constraint on (code, version). Lookup for new dunning episodes uses (operator_code, billing_mode, retired_at IS NULL) matching with * as wildcard.

level_definitions is the heart of the dunning program — an ordered JSON array describing each escalation step the customer goes through.

Level definition schema

Field	Type	Required	Notes
level	integer	yes	1-based level number. Levels MUST be contiguous (1, 2, 3, 4 — no gaps).
name	string	yes	Human-readable label shown on admin console and customer portal (e.g., WARNING, RESTRICTED, SUSPENDED, TERMINATED, SOFT_REMINDER).
grace_period_days	integer	yes	Days the Subscription stays at this level before advancing to the next. Counted from entered_level_at (the timestamp recorded when the Subscription first entered this level). Set to a small value (0-1) to advance almost immediately; set to a large value to keep the customer at this level for an extended period.
action_workflow_intent	enum	yes	One of: WARNING_ONLY (no workflow call, just emit event for notifications), RESTRICTION_ADD (call SUB-WF-RESTRICT-01 with ADD), SUSPEND_NP (call SUB-WF-SUSPEND-NP-01), TERMINATION (call SUB-WF-TERMINATE-01 with DUNNING_TERMINATION).
action_payload	object	yes (may be empty)	Payload for the workflow call. Shape depends on the intent — see below.
customer_portal_message_code	string	optional	A code the customer portal renders as a banner/notice at this level (the notification module owns the actual text per language).
next_action_visibility_days	integer	optional	If set, the customer portal shows "service will be {next action} in X days" when within this window of the next advance (gives customer warning).
payment_required_for_recovery	enum	optional	FULL (default — full debt must be cleared to recover) or MINIMUM_INSTALLMENT (a configurable percentage clears the dunning state but does not write off the remaining debt; the remaining stays as a fresh dunning episode at level 0 monitoring). MINIMUM_INSTALLMENT is V3.2 backlog — V3 supports only FULL.

action_payload shapes by intent

Intent	action_payload fields		
WARNING_ONLY	{} (empty — the Kafka event alone drives notification dispatch)		
RESTRICTION_ADD	{"restriction_codes": [<one or more restriction codes from SUB-LM-01's subscription_restriction catalog>]} — BIL-04 calls SUB-WF-RESTRICT-01 once per code in array order; each restriction gets originReason = "DUNNING_LEVEL_<N>" and systemManaged = true.		
SUSPEND_NP	{"reason_code": <code from SUB-WF-SUSPEND-NP-01's dunning reason set, e.g., DUNNING_GRACE_EXPIRED, DUNNING_WALLET_DEPLETED, DUNNING_ESCALATION_T3>}		
TERMINATION	`{"reason_code": <code, e.g., DUNNING_TERMINATION>, "equipment_disposition": <CUSTOMER_RETAINS \	PENDING_COLLECTION \	WRITTEN_OFF>}`

The intent and payload are pinned per program version. Adding new restriction types or new termination reasons requires a new program version (per R-BIL-04-C-1).

Recovery semantics derived from the level definitions

When a Subscription at level N reaches debt = 0, BIL-04 reverses the actions applied at levels 1..N in **reverse order**:

- **At level 1** (WARNING_ONLY): no reverse action needed — just emit SubscriptionDunningCleared.
- **At level 2** (RESTRICTION_ADD): call SUB-WF-RESTRICT-01 with REMOVE for each code in applied_restriction_codes array. Then emit cleared.
- **At level 3** (SUSPEND_NP): call SUB-WF-RESUME-01 with DUNNING_RESUME trigger. The resume workflow itself does NOT remove restrictions — BIL-04 first removes any dunning-applied restrictions (carryover from passing through level 2 on the way up), then triggers resume. Then emit cleared.
- **At level 4** (TERMINATION): no recovery possible — Subscription is gone.

This means the program author does NOT define explicit recovery actions; they're implied by the forward escalation. The applied_restriction_codes JSONB on dunning_state is BIL-04's bookkeeping for what to undo.

Worked example 1 — Wananchi Kenya, postpaid (data + TV + phone)

Wananchi Kenya offers a triple-play postpaid subscription (broadband data + TV + voice/phone). Restriction can hit any combination of the three services. Standard dunning policy gives a 7-day warning, then progressively restricts all three services together for 7 days, then suspends for 14 days, then a 30-day pre-termination review before final termination.

```
{
  "code": "wananchi_ke_postpaid_standard",
  "version": 1,
  "description": "Wananchi Kenya postpaid (data + TV + phone). 7-day warning, 7-day full-service restriction, 14-day suspe
  "operator_code": "WANANCHI_KE",
  "billing_mode": "POSTPAID",
  "pre_termination_review_required": true,
  "level_definitions": [
    {
      "level": 1,
      "name": "WARNING",
      "grace_period_days": 7,
      "action_workflow_intent": "WARNING_ONLY",
      "action_payload": {},
      "customer_portal_message_code": "DUN_WANANCHI_KE_L1_WARNING",
      "next_action_visibility_days": 7
    },
    {
      "level": 2,
      "name": "RESTRICTED",
      "grace_period_days": 7,
      "action_workflow_intent": "RESTRICTION_ADD",
      "action_payload": {
        "restriction_codes": [
          "DATA_THROTTLED_HIGH",
```

```
            "TV_CHANNELS_BLOCKED",
            "OUTGOING_VOICE_BARRED"
          ]
        },
        "customer_portal_message_code": "DUN_WANANCHI_KE_L2_RESTRICTED",
        "next_action_visibility_days": 7
      },
      {
        "level": 3,
        "name": "SUSPENDED",
        "grace_period_days": 14,
        "action_workflow_intent": "SUSPEND_NP",
        "action_payload": {
          "reason_code": "DUNNING_GRACE_EXPIRED"
        },
        "customer_portal_message_code": "DUN_WANANCHI_KE_L3_SUSPENDED",
        "next_action_visibility_days": 14
      },
      {
        "level": 4,
        "name": "TERMINATED",
        "grace_period_days": 30,
        "action_workflow_intent": "TERMINATION",
        "action_payload": {
          "reason_code": "DUNNING_TERMINATION",
          "equipment_disposition": "PENDING_COLLECTION"
        },
        "customer_portal_message_code": "DUN_WANANCHI_KE_L4_TERMINATED"
      }
    ]
  }
```

Why three restriction codes at level 2: each of the three Wananchi services (data, TV, phone) has its own dedicated restriction code per SUB-LM-01's `subscription_restriction` catalog. Applying all three together at level 2 means SUB-WF-RESTRICT-01 is called three times in sequence by BIL-04 — each restriction adds an entry to `applied_restriction_codes` on the dunning state. On recovery, all three are removed in reverse order via REMOVE workflow calls. The three-code-together pattern keeps dunning programs simple while preserving the catalog flexibility to throttle only one service in special operator-driven cases (which would use SUB-WF-RESTRICT-01 directly, outside the dunning flow).

Customer timeline (assume invoice due on Day 0, total owed 4,500 KES, customer doesn't pay):

Day	BIL-04 action	Subscription status	Customer-visible state
0	Invoice generated, due	ACTIVE	"Invoice due — please pay by Day 0"
1	BIL-02 emits `InvoiceOverdue` → BIL-04 creates dunning state at level 1	ACTIVE	"Payment overdue — service may be restricted in 7 days"
8	Scanner: grace expired, advance to level 2 → 3 sequential SUB-WF-RESTRICT-01 ADD calls (data, TV, voice)	ACTIVE (with restrictions on all 3 services)	"Service limited — pay 4,500 KES to restore data, TV, and voice"
15	Scanner: grace expired, advance to level 3 → call SUB-WF-SUSPEND-NP-01	SUSPENDED	"Service suspended for non-payment — pay 4,500 KES to restore"
29	Scanner: grace expired, pre-termination-review TRUE → transition to PENDING_TERMINATION_REVIEW	SUSPENDED	"Service will be permanently terminated soon — final reminder"
30-59	Operator reviews, may extend, clear, or confirm	SUSPENDED (no auto-advance)	(same)
59 (operator confirms)	Call SUB-WF-TERMINATE-01 with DUNNING_TERMINATION	TERMINATED	"Service terminated"

If customer pays on Day 10 (during level 2):

- BIL-04 receives `PaymentApplied` → debt = 0 → recovery path
- Removes all three restrictions via 3 sequential SUB-WF-RESTRICT-01 REMOVE calls (reverse order)
- Marks dunning state CLEARED, emits `SubscriptionDunningCleared`
- Subscription returns to fully ACTIVE

Worked example 2 — Wananchi Uganda, postpaid (data + TV + phone)

Wananchi Uganda offers the same triple-play postpaid service as Kenya. The dunning program is a near-clone, with two operator-driven differences: Uganda's regulator-suggested 10-day initial warning (slightly longer than KE's 7) and shorter pre-termination review (14 days vs KE's 30) because UG operators prefer faster churn cycles.

```
{
  "code": "wananchi_ug_postpaid_standard",
  "version": 1,
  "description": "Wananchi Uganda postpaid (data + TV + phone). 10-day warning per UG operator preference, 7-day restricti
  "operator_code": "WANANCHI_UG",
  "billing_mode": "POSTPAID",
  "pre_termination_review_required": true,
  "level_definitions": [
    {
      "level": 1,
      "name": "WARNING",
      "grace_period_days": 10,
      "action_workflow_intent": "WARNING_ONLY",
      "action_payload": {},
      "customer_portal_message_code": "DUN_WANANCHI_UG_L1_WARNING",
      "next_action_visibility_days": 10
    },
    {
      "level": 2,
      "name": "RESTRICTED",
      "grace_period_days": 7,
      "action_workflow_intent": "RESTRICTION_ADD",
      "action_payload": {
        "restriction_codes": [
          "DATA_THROTTLED_HIGH",
          "TV_CHANNELS_BLOCKED",
          "OUTGOING_VOICE_BARRED"
        ]
      },
      "customer_portal_message_code": "DUN_WANANCHI_UG_L2_RESTRICTED",
      "next_action_visibility_days": 7
    },
    {
      "level": 3,
      "name": "SUSPENDED",
      "grace_period_days": 14,
      "action_workflow_intent": "SUSPEND_NP",
      "action_payload": {
        "reason_code": "DUNNING_GRACE_EXPIRED"
      },
      "customer_portal_message_code": "DUN_WANANCHI_UG_L3_SUSPENDED",
      "next_action_visibility_days": 14
    },
    {
      "level": 4,
      "name": "TERMINATED",
      "grace_period_days": 14,
      "action_workflow_intent": "TERMINATION",
      "action_payload": {
        "reason_code": "DUNNING_TERMINATION",
        "equipment_disposition": "PENDING_COLLECTION"
      },
      "customer_portal_message_code": "DUN_WANANCHI_UG_L4_TERMINATED"
    }
  ]
}
```

Total cycle UG: 10 + 7 + 14 + 14 = 45 days, shorter than KE's 58.

Worked example 3 — Wananchi Tanzania, postpaid (data + TV + phone)

Tanzania regulator mandates a 14-day notice period before any service restriction on non-payment, and prefers lighter restrictions during the dunning period (voice should stay available for customer emergencies even during throttling). The TZ program reflects this with a longer initial warning, and a level-2 restriction that hits data and TV but leaves voice untouched.

```
{
  "code": "wananchi_tz_postpaid_standard",
  "version": 1,
  "description": "Wananchi Tanzania postpaid (data + TV + phone). Regulator-required 14-day notice; level 2 restriction om
  "operator_code": "WANANCHI_TZ",
  "billing_mode": "POSTPAID",
  "pre_termination_review_required": true,
  "level_definitions": [
    {
      "level": 1,
      "name": "WARNING",
      "grace_period_days": 14,
```

```
      "action_workflow_intent": "WARNING_ONLY",
      "action_payload": {},
      "customer_portal_message_code": "DUN_WANANCHI_TZ_L1_WARNING",
      "next_action_visibility_days": 14
    },
    {
      "level": 2,
      "name": "RESTRICTED",
      "grace_period_days": 14,
      "action_workflow_intent": "RESTRICTION_ADD",
      "action_payload": {
        "restriction_codes": [
          "DATA_THROTTLED_HIGH",
          "TV_CHANNELS_BLOCKED"
        ]
      },
      "customer_portal_message_code": "DUN_WANANCHI_TZ_L2_RESTRICTED",
      "next_action_visibility_days": 14
    },
    {
      "level": 3,
      "name": "SUSPENDED",
      "grace_period_days": 16,
      "action_workflow_intent": "SUSPEND_NP",
      "action_payload": {
        "reason_code": "DUNNING_GRACE_EXPIRED"
      },
      "customer_portal_message_code": "DUN_WANANCHI_TZ_L3_SUSPENDED"
    },
    {
      "level": 4,
      "name": "TERMINATED",
      "grace_period_days": 30,
      "action_workflow_intent": "TERMINATION",
      "action_payload": {
        "reason_code": "DUNNING_TERMINATION",
        "equipment_disposition": "PENDING_COLLECTION"
      },
      "customer_portal_message_code": "DUN_WANANCHI_TZ_L4_TERMINATED"
    }
  ]
}
```

Total cycle TZ: 14 + 14 + 16 + 30 = 74 days. Note voice is NOT in the level 2 restriction codes (regulator-driven). When suspension fires at level 3, the SUSPEND_NP workflow disables all services as one operation — at that point voice is no longer available either.

Worked example 4 — Yas Senegal fiber, pre-payment (data only)

Yas Senegal operates a **pre-payment** subscription model: the customer pays for each cycle before the cycle starts. This is NOT prepaid-wallet (no balance, no top-up, no consumption deduction). It is a subscription where each cycle is a fixed up-front charge that must be settled before the cycle activates.

The dunning trigger is `CyclePaymentMissed` emitted by BIL-03 at cycle boundary when the upcoming cycle has not been paid. Recovery semantics differ from postpaid: when the customer pays during dunning, BIL-01 activates a **fresh cycle starting from the payment date** — the missed cycle is NOT retroactively billed and the customer does not get service for the suspended period (per R-BIL-04-R-6).

The dunning program references only the data-throttle restriction code — there is no TV or phone restriction in Yas SN's `subscription_restriction` catalog. The included telco SIM pack add-on rides on the fiber subscription's dunning state automatically — when the fiber's data is throttled or suspended, the SIM pack add-on becomes correspondingly limited via the same fulfillment chain, without BIL-04 needing to track the add-on separately.

```
{
  "code": "yas_sn_fiber_prepayment",
  "version": 1,
  "description": "Yas Senegal fiber data, pre-payment subscription model. Customer pays each cycle before it starts. Trigg
  "operator_code": "YAS_SN",
  "billing_mode": "PREPAYMENT",
  "pre_termination_review_required": true,
  "level_definitions": [
    {
      "level": 1,
      "name": "WARNING",
      "grace_period_days": 5,
      "action_workflow_intent": "WARNING_ONLY",
      "action_payload": {},
      "customer_portal_message_code": "DUN_YAS_SN_L1_WARNING",
      "next_action_visibility_days": 5
```

```
      },
      {
        "level": 2,
        "name": "RESTRICTED",
        "grace_period_days": 5,
        "action_workflow_intent": "RESTRICTION_ADD",
        "action_payload": {
          "restriction_codes": ["DATA_THROTTLED_HIGH"]
        },
        "customer_portal_message_code": "DUN_YAS_SN_L2_RESTRICTED",
        "next_action_visibility_days": 5
      },
      {
        "level": 3,
        "name": "SUSPENDED",
        "grace_period_days": 21,
        "action_workflow_intent": "SUSPEND_NP",
        "action_payload": {
          "reason_code": "DUNNING_CYCLE_UNPAID"
        },
        "customer_portal_message_code": "DUN_YAS_SN_L3_SUSPENDED",
        "next_action_visibility_days": 21
      },
      {
        "level": 4,
        "name": "TERMINATED",
        "grace_period_days": 21,
        "action_workflow_intent": "TERMINATION",
        "action_payload": {
          "reason_code": "DUNNING_TERMINATION",
          "equipment_disposition": "PENDING_COLLECTION"
        },
        "customer_portal_message_code": "DUN_YAS_SN_L4_TERMINATED"
      }
    ]
  }
```

Total cycle Yas SN: 5 + 5 + 21 + 21 = 52 days from missed-cycle-start to potential termination.

Customer timeline (assume cycle was due to start Day 0, customer didn't pay):

Day	BIL-04 action	Subscription status	Customer-visible state
-3	Cycle reminder pre-notice (notification module)	ACTIVE (prior cycle running)	"Your cycle starts in 3 days — pay now to ensure continuous service"
0	BIL-03 detects unpaid cycle at boundary, emits `CyclePaymentMissed` → BIL-04 creates dunning state at level 1	ACTIVE	"Cycle unpaid — please pay to continue service"
5	Scanner: grace expired, advance to level 2 → call SUB-WF-RESTRICT-01 ADD `DATA_THROTTLED_HIGH`	ACTIVE (with restriction)	"Service throttled — pay to restore full speed"
10	Scanner: grace expired, advance to level 3 → call SUB-WF-SUSPEND-NP-01	SUSPENDED	"Service suspended — pay to restore"
31	Scanner: grace expired, pre-termination-review TRUE → PENDING_TERMINATION_REVIEW	SUSPENDED	"Service will be terminated soon — final reminder"
31-52	Operator reviews	SUSPENDED	(same)
52 (operator confirms)	Call SUB-WF-TERMINATE-01 with DUNNING_TERMINATION	TERMINATED	"Service terminated"

If customer pays on Day 8 (during level 2):

- BIL-04 receives `PaymentApplied` → debt = 0 → recovery path
- Removes `DATA_THROTTLED_HIGH` via SUB-WF-RESTRICT-01 REMOVE
- BIL-01 (independently) aligns the cycle: **new cycle starts Day 8**, NOT Day 0. The customer pays for one cycle starting today; the period Day 0–7 is not retroactively billed and was not actually serviced (throttled).
- Marks dunning state CLEARED, emits `SubscriptionDunningCleared`
- Subscription returns to fully ACTIVE on the new cycle anchor

Why only one restriction code: Yas Senegal fiber offers data only. The `subscription_restriction` catalog for the YAS_SN tenant contains only data-related codes (`DATA_THROTTLED_LOW`, `DATA_THROTTLED_HIGH`). TV and voice

restriction codes are absent from the YAS_SN catalog. SUB-WF-RESTRICT-01's R-C-1 rule rejects unknown codes — so a misconfigured program referencing `TV_CHANNELS_BLOCKED` for Yas SN would fail validation at the workflow boundary, surfacing the mistake early.

SIM pack add-on cascade: when the fiber subscription's data is throttled at level 2, the included telco SIM pack add-on becomes correspondingly limited via the standard fulfillment chain (FUL-04 issues the network commands that affect both the fiber service and the linked SIM pack). At suspension (level 3), both the fiber service and the SIM pack add-on are disabled. BIL-04 does NOT track the add-on separately — the dunning state is on the parent subscription, and the add-on's service-level changes follow automatically.

Shorter grace periods than postpaid: pre-payment subscriptions typically have shorter grace periods at the front of the dunning cycle than postpaid (5 days vs 10–14 for Wananchi). Rationale: in pre-payment the customer's non-payment is an unambiguous signal of intent to discontinue (or oversight that's quickly remedied with a small bill paid). In postpaid the customer has already consumed service and an invoice dispute or temporary cash flow issue is more common; longer grace is warranted.

Worked example 5 — Wananchi Kenya, PREPAID wallet (data + TV + phone)

Most Wananchi Kenya customers are on the **PREPAID wallet** model. Customer tops up their wallet anytime (via mobile money, bank, retail agents — owned by BIL-05). At each cycle boundary, BIL-03 queries BIL-05 for wallet balance and if sufficient, debits the wallet for the cycle charge. If insufficient, BIL-03 emits `CyclePaymentMissed` and BIL-04 enters dunning.

The PREPAID dunning recovery path is unique: BIL-04 listens for `WalletToppedUp` events from BIL-05. The moment a topup brings the wallet balance high enough to cover the cycle, BIL-04 automatically debits the wallet (via BIL-05), advances the cycle anchor (via BIL-03), removes any dunning-applied restrictions, and emits `SubscriptionDunningCleared` — all without operator intervention. This is per R-BIL-04-R-8.

```
{
  "code": "wananchi_ke_prepaid_standard",
  "version": 1,
  "description": "Wananchi Kenya PREPAID wallet (data + TV + phone). Triggered on CyclePaymentMissed when wallet insuffici
  "operator_code": "WANANCHI_KE",
  "billing_mode": "PREPAID",
  "pre_termination_review_required": true,
  "level_definitions": [
    {
      "level": 1,
      "name": "WARNING",
      "grace_period_days": 3,
      "action_workflow_intent": "WARNING_ONLY",
      "action_payload": {},
      "customer_portal_message_code": "DUN_WANANCHI_KE_PP_L1_WARNING",
      "next_action_visibility_days": 3
    },
    {
      "level": 2,
      "name": "RESTRICTED",
      "grace_period_days": 4,
      "action_workflow_intent": "RESTRICTION_ADD",
      "action_payload": {
        "restriction_codes": [
          "DATA_THROTTLED_HIGH",
          "TV_CHANNELS_BLOCKED",
          "OUTGOING_VOICE_BARRED"
        ]
      },
      "customer_portal_message_code": "DUN_WANANCHI_KE_PP_L2_RESTRICTED",
      "next_action_visibility_days": 4
    },
    {
      "level": 3,
      "name": "SUSPENDED",
      "grace_period_days": 14,
      "action_workflow_intent": "SUSPEND_NP",
      "action_payload": {
        "reason_code": "DUNNING_WALLET_INSUFFICIENT"
      },
      "customer_portal_message_code": "DUN_WANANCHI_KE_PP_L3_SUSPENDED",
      "next_action_visibility_days": 14
    },
    {
      "level": 4,
      "name": "TERMINATED",
      "grace_period_days": 21,
      "action_workflow_intent": "TERMINATION",
      "action_payload": {
```

```
        "reason_code": "DUNNING_TERMINATION",
        "equipment_disposition": "PENDING_COLLECTION"
      },
      "customer_portal_message_code": "DUN_WANANCHI_KE_PP_L4_TERMINATED"
    }
  ]
}
```

Total cycle for Wananchi KE PREPAID: 3 + 4 + 14 + 21 = 42 days from missed cycle boundary to potential termination. Shorter than KE postpaid (58 days) because the wallet model means non-payment is a clear customer signal (similar reasoning to PREPAYMENT — see comparison summary).

Customer timeline (cycle was due to start Day 0, wallet had insufficient balance):

Day	BIL-04 action	Subscription status	Customer-visible state
-3	BIL-05 emits low-balance warning (via notification module)	ACTIVE (current cycle running)	"Wallet low — top up X KES before [date] to ensure continuous service"
0	BIL-03 detects insufficient wallet at boundary, emits `CyclePaymentMissed` → BIL-04 creates dunning state at level 1	ACTIVE	"Wallet insufficient — top up to activate service"
3	Scanner: grace expired, advance to level 2 → 3 sequential SUB-WF-RESTRICT-01 ADD calls	ACTIVE (with restrictions)	"Service restricted — top up X KES to restore"
7	Scanner: grace expired, advance to level 3 → call SUB-WF-SUSPEND-NP-01	SUSPENDED	"Service suspended — top up X KES to restore"
21	Scanner: grace expired, pre-termination-review TRUE → PENDING_TERMINATION_REVIEW	SUSPENDED	"Service will be terminated soon — final reminder"
21-42	Operator reviews	SUSPENDED	(same)
42 (operator confirms)	Call SUB-WF-TERMINATE-01 with DUNNING_TERMINATION	TERMINATED	"Service terminated"

Auto-recovery on topup (assume customer tops up on Day 5 during level 2):

Step	Action
Day 5 11:00	Customer tops up 5000 KES via mobile money → BIL-05 records → BIL-05 emits `WalletToppedUp`
Day 5 11:00:01	BIL-04 consumer receives event, queries BIL-05 for new balance (5000 KES) and current cycle charge (4500 KES)
Day 5 11:00:02	Balance sufficient → BIL-04 calls BIL-05 debit (4500 KES, wallet now 500 KES), calls BIL-03 to advance cycle anchor
Day 5 11:00:03	BIL-04 calls SUB-WF-RESTRICT-01 REMOVE for all 3 restrictions
Day 5 11:00:04	All restrictions removed → BIL-04 marks dunning state CLEARED, emits `SubscriptionDunningCleared`
Day 5 11:00:05	Subscription returns to fully ACTIVE on new cycle anchor starting Day 5

Customer experience: a top-up restores service in under 30 seconds. The customer experience is similar to PREPAYMENT recovery (R-6) but with a wallet-based mechanism. Architecturally distinct, behaviorally similar.

Insufficient topup (assume customer tops up only 2000 KES on Day 4 — not enough for the 4500 KES cycle):

Step	Action
Day 4	Customer tops up 2000 KES → BIL-05 emits `WalletToppedUp`
Day 4	BIL-04 queries balance (2000 KES) and cycle charge (4500 KES) — insufficient
Day 4	BIL-04 emits `SubscriptionDunningDebtIncreased` with corrected shortfall (2500 KES still needed); stays at current level
(customer notification fires next)	"You topped up 2000 KES. Top up 2500 KES more to restore service."

Comparison summary across the five examples

Program	Operator	Billing model	Services	Levels	Total cycle	Trigger event	Recovery mechanism
`wananchi_ke_postpaid_standard v1`	Wananchi KE	POSTPAID	data + TV + phone	4	~58 days	`InvoiceOverdue`	`PaymentApplied` settles overdue debt, continues existing cycle
`wananchi_ug_postpaid_standard v1`	Wananchi UG	POSTPAID	data + TV + phone	4	~45 days	`InvoiceOverdue`	Same as KE postpaid (faster grace cycle)
`wananchi_tz_postpaid_standard v1`	Wananchi TZ	POSTPAID	data + TV + phone	4	~74 days	`InvoiceOverdue`	Same as KE postpaid (regulator-driven longer cycle, voice preserved at L2)
`wananchi_ke_prepaid_standard v1`	Wananchi KE	PREPAID	data + TV + phone	4	~42 days	`CyclePaymentMissed` (wallet-insufficient)	`WalletToppedUp` auto-debits wallet, advances cycle anchor, restores service
`yas_sn_fiber_prepayment v1`	Yas SN	PREPAYMENT	data only	4	~52 days	`CyclePaymentMissed` (cycle-invoice-unpaid)	`PaymentApplied` for the cycle invoice starts a fresh cycle from payment date

Patterns to take away:

1. **All five programs run a 4-level structural shape** (WARN → RESTRICT → SUSPEND → review-then-TERMINATE) — operational consistency across operators and billing models.
2. **Each operator/mode combination tunes the grace periods independently** — KE postpaid 58 days, UG postpaid 45 days, TZ postpaid 74 days (regulator), KE prepaid 42 days, Yas SN prepayment 52 days. Per-operator + per-billing-mode versioning gives clean separation.
3. **Restriction codes per program follow the services offered** — Wananchi (data + TV + voice) hits all three; Wananchi TZ excludes voice at L2 per regulator; Yas SN data only.
4. `pre_termination_review_required = true` **across all five** — operator-preferred safeguard for all V3 subscriptions before permanent termination.
5. **Three billing models in V3, same dunning shape, different recovery mechanisms**: POSTPAID (overdue debt → pay invoice → continue cycle), PREPAID (wallet insufficient → wallet topup → auto-debit → advance cycle), PREPAYMENT (cycle invoice unpaid → pay invoice → fresh cycle from payment date). The dunning state machine and escalation logic are identical; only the trigger source and the recovery handler differ.
6. **PREPAID auto-recovery is the most customer-friendly path** — a topup triggers wallet-debit + cycle-advance + restriction-removal + suspension-resume in seconds, with no operator involvement. Achievable because BIL-04 listens for `WalletToppedUp` Kafka events from BIL-05 in real time (no scanner delay).
7. **PREPAID and PREPAYMENT grace periods are shorter than POSTPAID** — non-payment in wallet/pre-cycle models is a clear customer signal (or quickly remedied), while in postpaid invoice disputes and cash flow issues are more common and longer grace is warranted.
8. **Level structure is consistent (4 levels) but contents differ** — the schema's flexibility allows per-operator and per-billing-model variation without code changes, just data variation.
9. **Add-ons follow the parent subscription's dunning state automatically** — Yas SN's included telco SIM pack does not need its own dunning logic; restrictions and suspension cascade through the standard fulfillment chain.
10. **V3 supports all three billing models on day 1** — POSTPAID (some Wananchi customers), PREPAID (majority of Wananchi customers — wallet/topup-based), PREPAYMENT (Yas SN — per-cycle invoice paid before cycle).

`dunning_state` — active dunning episodes

Column	Type	Notes
`id`	TEXT PK	ULID
`subscription_id`	TEXT NOT NULL UNIQUE	One active dunning state per Subscription
`dunning_program_ref`	TEXT NOT NULL	References `dunning_program.code`

Column	Type	Notes
dunning_program_version	INTEGER NOT NULL	Pinned at entry — version stays for the lifetime of this dunning episode
dunning_level	INTEGER NOT NULL	0 = cleared/none, 1-4 = active levels
entered_dunning_at	TIMESTAMPTZ NOT NULL	When this episode started (level 1 first entered)
entered_level_at	TIMESTAMPTZ NOT NULL	When the Subscription entered the **current** level (advance points reset this)
outstanding_debt_amount	NUMERIC(15,2) NOT NULL	
outstanding_debt_currency	TEXT NOT NULL	
triggering_event_type	TEXT NOT NULL	INVOICE_OVERDUE (postpaid), CYCLE_PAYMENT_MISSED (PREPAID wallet-insufficient OR PREPAYMENT cycle-unpaid — payload distinguishes via billing_mode) — records what triggered this dunning episode
triggering_event_ref	TEXT NOT NULL	Source identifier — invoice ID for INVOICE_OVERDUE, cycle reference for CYCLE_PAYMENT_MISSED (with billing_mode_at_trigger snapshot to know if it was PREPAID wallet or PREPAYMENT invoice)
applied_restriction_codes	JSONB NOT NULL DEFAULT '[]'	Array of restriction codes BIL-04 has applied (for recovery)
last_evaluated_at	TIMESTAMPTZ NULL	
next_evaluation_at	TIMESTAMPTZ NULL	NULL means paused (SUSPENDED_BY_PAUSE)
status	TEXT NOT NULL	ACTIVE / CLEARED / SUSPENDED_BY_PAUSE / PENDING_TERMINATION_REVIEW / RECOVERY_FAILED / ARCHIVED
last_workflow_failure_code	TEXT NULL	Set when a workflow call fails
last_workflow_failure_at	TIMESTAMPTZ NULL	
workflow_failure_attempts	INTEGER NOT NULL DEFAULT 0	Retry counter, resets on success
created_at	TIMESTAMPTZ NOT NULL	
cleared_at	TIMESTAMPTZ NULL	Set when status → CLEARED
archived_at	TIMESTAMPTZ NULL	Set when row moved to archive

Indexes:

- (status, next_evaluation_at) — scanner query
- (subscription_id) UNIQUE — single active state per Subscription
- (dunning_level, entered_level_at) — admin console aggregation

_audit shadow table per FOUNDATION_DB.md.

dunning_state_archive

Same columns as dunning_state plus archive_reason (CLEARED_FULLY_PAID / TERMINATED / ADMIN_CLEARED / SUPERSEDED). Monthly partitioned. P10Y retention.

Rules (V3 in code, V3.2 may use Drools)

In V3, dunning evaluation is straightforward branching logic in DunningEvaluationService:

```
evaluate(state):
  if state.status != ACTIVE: skip

  refresh debt amount from BIL-01

  if debt == 0:
    trigger recovery (level-specific path)
    return

  current_level_def = program.levels[state.dunning_level]
  grace_elapsed = now - state.entered_level_at >= current_level_def.grace_period_days

  if not grace_elapsed:
    update next_evaluation_at, return

  if state.dunning_level + 1 > MAX_LEVEL:
```

```
  no further advance possible (already at 4)
  return

next_level_def = program.levels[state.dunning_level + 1]

if next_level_def.level == 4 AND program.pre_termination_review_required:
  transition to PENDING_TERMINATION_REVIEW, emit event
  return

invoke next_level_def.action_workflow_intent
on success: state.dunning_level += 1, state.entered_level_at = now
on failure: state.last_workflow_failure_*, attempts++
```

This is documented as code, not DRL, to keep V3 simple. V3.2 may move conditional logic to DRL if operators need to define custom dunning conditions without code changes (e.g., "skip level 2 for high-VIP customers" — feasibility unclear at this stage).

Implementation

The core service class follows the same shape pattern as workflow services per SUB-WF-FRAMEWORK-01 (idempotency, try/catch revert) but for dunning state mutations rather than Subscription state mutations.

```
@Service
public class DunningEvaluationService {

    @Transactional
    public DunningEvaluationResult evaluateOne(String dunningStateId) {
        DunningState state = stateRepo.lockById(dunningStateId);  // SELECT FOR UPDATE
        if (state.getStatus() != DunningStateStatus.ACTIVE) {
            return DunningEvaluationResult.skipped(state, "not_active");
        }

        DunningProgram program = programRepo.findByCodeAndVersion(state.getDunningProgramRef(), state.getDunningProgramVer
        BigDecimal currentDebt = billingDebtApi.fetchOutstandingDebt(state.getSubscriptionId(), state.getOutstandingDebtCu
        state.setOutstandingDebtAmount(currentDebt);

        if (currentDebt.compareTo(BigDecimal.ZERO) == 0) {
            return recoveryService.recoverFromLevel(state);
        }

        DunningLevelDefinition currentLevelDef = program.levelAt(state.getDunningLevel());
        if (!gracePeriodElapsed(state, currentLevelDef)) {
            scheduleNextEvaluation(state, currentLevelDef);
            return DunningEvaluationResult.held(state, "grace_active");
        }

        int nextLevel = state.getDunningLevel() + 1;
        if (nextLevel > program.maxLevel()) {
            return DunningEvaluationResult.held(state, "already_at_max");
        }
        DunningLevelDefinition nextLevelDef = program.levelAt(nextLevel);

        if (nextLevelDef.getLevel() == 4 && program.isPreTerminationReviewRequired()) {
            state.setStatus(DunningStateStatus.PENDING_TERMINATION_REVIEW);
            stateRepo.save(state);
            eventPublisher.publishTerminationPending(state);
            return DunningEvaluationResult.pendingReview(state);
        }

        try {
            workflowInvoker.invoke(state, nextLevelDef);
            state.setDunningLevel(nextLevel);
            state.setEnteredLevelAt(Instant.now());
            state.setWorkflowFailureAttempts(0);
            state.setLastWorkflowFailureCode(null);
            scheduleNextEvaluation(state, nextLevelDef);
            stateRepo.save(state);
            eventPublisher.publishAdvanced(state, nextLevelDef);
            return DunningEvaluationResult.advanced(state, nextLevelDef);
        } catch (WorkflowInvocationFailure f) {
            state.setLastWorkflowFailureCode(f.getCode());
            state.setLastWorkflowFailureAt(Instant.now());
            state.setWorkflowFailureAttempts(state.getWorkflowFailureAttempts() + 1);
            if (state.getWorkflowFailureAttempts() >= 3) {
                state.setStatus(DunningStateStatus.RECOVERY_FAILED);
                eventPublisher.publishRecoveryFailed(state, f);
            } else {
                scheduleRetry(state);
            }
            stateRepo.save(state);
            return DunningEvaluationResult.failed(state, f);
```

```
            }
        }
    }
```

The Laravel/PHP equivalent follows the same logic with `DB::transaction()` closure and Throwable handling, per the dual-stack pattern in SUB-WF-FRAMEWORK-01.

Events (Kafka)

`sophix.billing.dunning.entered`

Example for a Wananchi KE postpaid subscription:

```
{
  "eventType": "SubscriptionEnteredDunning",
  "aggregateId": "dunning_state_01HX...",
  "timestamp": "2026-07-15T00:00:00Z",
  "payload": {
    "dunningStateId": "dunning_state_01HX...",
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "operatorCode": "WANANCHI_KE",
    "billingMode": "POSTPAID",
    "dunningProgramRef": "wananchi_ke_postpaid_standard",
    "dunningProgramVersion": 1,
    "enteredDunningAt": "2026-07-15T00:00:00Z",
    "dunningLevel": 1,
    "outstandingDebtAmount": 4500.00,
    "outstandingDebtCurrency": "KES",
    "triggeringEventType": "INVOICE_OVERDUE",
    "triggeringEventRef": "inv_01HX...",
    "nextEvaluationAt": "2026-07-22T00:00:00Z"
  }
}
```

Example for a Yas SN pre-payment subscription:

```
{
  "eventType": "SubscriptionEnteredDunning",
  "aggregateId": "dunning_state_01HY...",
  "timestamp": "2026-08-01T00:00:00Z",
  "payload": {
    "dunningStateId": "dunning_state_01HY...",
    "subscriptionId": "sub_01HY...",
    "customerId": "cust_01HY...",
    "operatorCode": "YAS_SN",
    "billingMode": "PREPAYMENT",
    "dunningProgramRef": "yas_sn_fiber_prepayment",
    "dunningProgramVersion": 1,
    "enteredDunningAt": "2026-08-01T00:00:00Z",
    "dunningLevel": 1,
    "outstandingDebtAmount": 25000.00,
    "outstandingDebtCurrency": "XOF",
    "triggeringEventType": "CYCLE_PAYMENT_MISSED",
    "triggeringEventRef": "cycle_2026_08_sub_01HY...",
    "nextEvaluationAt": "2026-08-06T00:00:00Z"
  }
}
```

`sophix.billing.dunning.advanced`

```
{
  "eventType": "SubscriptionDunningAdvanced",
  "aggregateId": "dunning_state_01HX...",
  "timestamp": "2026-07-22T00:00:00Z",
  "payload": {
    "dunningStateId": "dunning_state_01HX...",
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "fromLevel": 1,
    "toLevel": 2,
    "actionTaken": "RESTRICTION_ADD",
    "actionPayload": {
      "restrictionCodes": ["OUTGOING_VOICE_BARRED", "DATA_THROTTLED_HIGH"]
    },
    "outstandingDebtAmount": 4500.00,
    "outstandingDebtCurrency": "KES",
    "advancedAt": "2026-07-22T00:00:00Z",
    "nextEvaluationAt": "2026-07-29T00:00:00Z"
  }
}
```

`sophix.billing.dunning.debtincreased`

```
{
  "eventType": "SubscriptionDunningDebtIncreased",
  "aggregateId": "dunning_state_01HX...",
  "timestamp": "2026-07-20T00:00:00Z",
  "payload": {
    "dunningStateId": "dunning_state_01HX...",
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "priorDebtAmount": 4500.00,
    "newDebtAmount": 6750.00,
    "currency": "KES",
    "currentLevel": 1,
    "triggeringInvoiceRef": "inv_01HY..."
  }
}
```

`sophix.billing.dunning.cleared`

```
{
  "eventType": "SubscriptionDunningCleared",
  "aggregateId": "dunning_state_01HX...",
  "timestamp": "2026-08-05T12:30:00Z",
  "payload": {
    "dunningStateId": "dunning_state_01HX...",
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "clearedFromLevel": 3,
    "totalDebtSettled": 4500.00,
    "currency": "KES",
    "clearedDuration_days": 21,
    "recoveryAction": "RESUME_FROM_SUSPENSION",
    "clearedAt": "2026-08-05T12:30:00Z"
  }
}
```

`sophix.billing.dunning.recoveryfailed`

```
{
  "eventType": "SubscriptionDunningRecoveryFailed",
  "aggregateId": "dunning_state_01HX...",
  "timestamp": "2026-07-30T12:30:00Z",
  "payload": {
    "dunningStateId": "dunning_state_01HX...",
    "subscriptionId": "sub_01HX...",
    "customerId": "cust_01HX...",
    "failureType": "WORKFLOW_REPEATEDLY_FAILED",
    "failedAction": "SUSPEND_NP",
    "failureReasonCode": "FULFILLMENT_TIMEOUT",
    "failureReasonDetail": "FUL-04 unavailable across 3 attempts",
    "attempts": 3,
    "requiresOperatorAttention": true
  }
}
```

`sophix.billing.dunning.terminationpending`

Emitted when a postpaid dunning state reaches level 4 and `pre_termination_review_required = true`. Operations team receives an alert; CSR queue surfaces this for manual confirmation.

`sophix.billing.dunning.adminoverride`

Audited every time an admin uses an override endpoint (hold, advance, clear-without-payment, force-terminate, extend-review).

APIs (in this module)

Read endpoints

```
GET /api/dunning-states/{subscriptionId}                    ← current dunning state of a Subscription
GET /api/dunning-states/{subscriptionId}/history            ← full history including archived
GET /api/dunning-states?level={n}&status={s}&limit=50       ← admin console listing
GET /api/dunning-states/pending-termination-review          ← admin queue
GET /api/dunning-programs                                    ← catalog list
GET /api/dunning-programs/{code}?version={v}                ← specific program version
```

Admin override endpoints (role `DUNNING_ADMIN`)

```
POST /api/dunning-states/{id}/hold                           ← postpone next evaluation
POST /api/dunning-states/{id}/advance                        ← force advance to next level (skip grace)
POST /api/dunning-states/{id}/clear-without-payment          ← write off debt, recover Subscription
```

```
POST /api/dunning-states/{id}/force-terminate                ← bypass pre-termination-review
POST /api/dunning-states/{id}/extend-review                  ← push pending-termination-review out
POST /api/dunning-states/{id}/admin-clear-restrictions       ← lift dunning-applied restrictions outside normal recove
```

Each requires `reason` field in the body, audited via `DunningAdminOverride` Kafka event.

Catalog admin (role `DUNNING_ADMIN`)

```
POST /api/dunning-programs/{code}/new-version                ← publishes a new version, retires the prior
POST /api/dunning-programs                                    ← creates a brand-new program code
```

(No DELETE — programs are versioned and superseded, never deleted.)

Internal endpoints (called by other modules)

```
POST /api/dunning-states/refresh-debt                         ← called by BIL-01 on PaymentApplied/ChargeApplied (alte
```

Cross-DD coordination

Touched	What BIL-04 owns	What the other DD owns
SUB-WF-FRAMEWORK-01	Follows synchronous service-class pattern; uses scheduled scanner pattern.	Owns the framework.
SUB-WF-RESTRICT-01	Calls with `ADD` intent for level 2 entry; calls with `REMOVE` intent for recovery. Idempotency keys include dunning episode markers so retries are safe.	Applies/lifts restrictions, validates rules, emits restriction events.
SUB-WF-SUSPEND-NP-01	Calls for level 3 entry. Passes `dunningReasonCode`, `dunningCycleReference`, `dunningEscalationLevel`, `outstandingDebtAmount`.	Suspends Subscription, emits `SubscriptionSuspendedForNonPayment`.
SUB-WF-RESUME-01 (DUNNING_RESUME trigger)	Calls for recovery from level 3.	Resumes Subscription, validates trigger gating.
SUB-WF-TERMINATE-01 (DUNNING_TERMINATION trigger)	Calls for level 4 (after pre-termination-review confirmation if required). Passes `dunningEscalationLevel`, equipment disposition from program config.	Terminates Subscription, emits settlement event.
BIL-01 / BIL-02	Consumes `InvoiceOverdue` (BIL-02, postpaid) and `PaymentApplied` (BIL-01, postpaid + pre-payment recovery) Kafka events. Queries outstanding debt via REST during postpaid evaluation. BIL-01 handles cycle realignment on payment for pre-payment subscriptions independently (BIL-04 does not coordinate this).	Owns invoice/charge accounting and cycle alignment; emits events.
BIL-03	Consumes `CyclePaymentMissed` (PREPAID wallet-insufficient AND PREPAYMENT cycle-unpaid — payload distinguishes via `billing_mode`). Calls BIL-03's internal `advance-cycle` endpoint after PREPAID wallet-topup auto-recovery (R-BIL-04-R-8).	Owns cycle close logic and boundary detection. Owns cycle anchor advance API called by BIL-04 on PREPAID recovery.
BIL-05 (Wallet & Topup)	Consumes `WalletToppedUp` events for PREPAID auto-recovery (R-BIL-04-R-8). Calls BIL-05 to query wallet balance and to debit wallet on auto-recovery.	Owns the wallet, topup channels, balance tracking, debit/credit operations. Emits `WalletToppedUp` events on every successful topup.
SUB-LM-01	Consumes `SubscriptionPaused` and `SubscriptionResumed` (voluntary) events for the SUSPENDED_BY_PAUSE handling.	Owns Subscription state.
BIL-CFG-01	None directly — dunning programs are BIL-04's own catalog. BillableEvents for reactivation fees referenced in dunning programs are owned by BIL-CFG-01 (e.g., `RECONNECTION_FEE` BillableEvent) and triggered via the standard workflow billing call.	Owns BillableEvents.
`notification` **module (DD_NOT-01, planned)**	Emits 7 dunning event types with all fields a notification template needs: dunning level, debt amount, next escalation date, recovery action taken. Workflow does not call notification directly.	Consumes the events. Expected customer comms: payment-reminder SMS + email at level 1 entry and on debt-increase, service-restriction notice at level 2, suspension-with-payment-instructions at level 3, termination-warning at pending-termination-review, termination-confirmation at level 4. Templates per tenant, customer language and channel preferences honored.

Dependencies

Dependency	Owner	Status
SUB-WF-FRAMEWORK-01	subscription team	✓ DD complete (v0.9)
SUB-WF-RESTRICT-01	subscription team	✓ DD complete (v0.9)
SUB-WF-SUSPEND-NP-01	subscription team	✓ DD complete (v0.9)
SUB-WF-RESUME-01 (DUNNING_RESUME trigger)	subscription team	✓ DD complete (v0.9)
SUB-WF-TERMINATE-01 (DUNNING_TERMINATION trigger)	subscription team	✓ DD complete (v0.9)
BIL-01 (`PaymentApplied` event, debt query API)	billing team	■ DD partial (v0.8 — debt query API not yet specified)
BIL-02 (`InvoiceOverdue` event)	billing team	■ pending DD
BIL-03 (`CyclePaymentMissed` event, pre-payment cycle alignment on `PaymentApplied`, internal advance-cycle-on-recovery endpoint for PREPAID auto-recovery)	billing team	✓ DD complete (v1.1)
BIL-05 Wallet & Topup (wallet balance query, debit, `WalletToppedUp` event)	billing team	■ pending DD (Wave 1 backlog)
FOUNDATION_DB.md	infra team	✓ Documented
FOUNDATION_KAFKA.md	infra team	✓ Documented
FOUNDATION_AUTH.md (`DUNNING_ADMIN`, `DUNNING_OPERATOR`, `BILLING_INTERNAL` roles)	infra team	✓ Documented
DD_NOT-01 Notification Service	notification team	■ pending DD (Wave 1 backlog)

B — Current TZ

To be filled in after codebase review. No claims about what the TZ-deployed Sophix codebase does for dunning are recorded here until the codebase has been read directly.

The cartography phase (per the pending plan) will produce this content. Once the codebase is available, this section will be filled with verified statements about which dunning-related capabilities exist and which do not — replacing this placeholder.

C — V3 design rationale

This DD's V3 design choices follow from the workshop requirements (section A) and the per-tenant + per-billing-mode variation surface (5 example programs in worked examples 1-5 covering Wananchi KE/UG/TZ postpaid, Yas SN PREPAYMENT, Wananchi KE PREPAID). The design intent:

What V3 introduces

1. **Dedicated `billing-dunning` module** with an explicit state machine. Single source of truth for "where is this customer in the dunning lifecycle".
2. **Per-Subscription `dunning_state` row** as the authoritative record. Independent across multiple Subscriptions for the same customer (customer-level dunning aggregation is V3.2 backlog).
3. **Multi-step configurable escalation ladder** — warning → restricted → suspended → terminated — with arbitrary intermediate levels per `dunning_program` definition.
4. **Per-tenant, per-billing-mode dunning policy catalog** with versioning. Each operator + billing mode gets its own `dunning_program` (Wananchi KE postpaid, Wananchi UG postpaid, Wananchi TZ postpaid, Wananchi KE prepaid, Yas SN PREPAYMENT, etc.). In-flight episodes pin their version so live config changes don't disrupt ongoing escalations.
5. **Automatic recovery on payment** — no manual operator action needed for common cases. Recovery is event-driven via SUB-WF-RESUME-01 and SUB-WF-RESTRICT-01.
6. **Pre-termination review** gating (configurable per program; default TRUE for postpaid) to prevent accidental terminations.
7. **Decoupled event-driven coordination** with notification, CVM, analytics modules. Dunning events drive notification module (DD_NOT-01).
8. **Pause-aware dunning** (SUSPENDED_BY_PAUSE) so voluntary pause does not trigger dunning advances against the subscription.

9. **Clear separation from voluntary suspension** — different status SUSPENDED + `current_transition_type = SUSPEND_NP` + dunning state row owned by BIL-04. No overloading of statuses.
10. **Structured admin override endpoints** with audit (`DunningAdminOverride` event).
11. **Workflow confirmation via Kafka events** — BIL-04 verifies its workflow calls actually committed.
12. **Recovery-failed handling** with retry-then-flag-for-operator.
13. **Customer portal visibility** of dunning state.
14. **P10Y audit trail** via shadow tables + archive table + Kafka events (7y retention on event stream).
15. **Restriction codes per program follow the services offered** — Wananchi (data + TV + voice) hits all three; Wananchi TZ excludes voice at L2 per Tanzania regulator preference; Yas SN data only.

Migration approach (TZ-baseline-agnostic)

1. Deploy `billing-dunning` module with its three tables on cluster C.
2. Seed `dunning_program` with one program per operator + billing-mode combination: `wananchi_ke_postpaid_standard`, `wananchi_ke_prepaid_standard`, `wananchi_ug_postpaid_standard`, `wananchi_ug_prepaid_standard`, `wananchi_tz_postpaid_standard`, `wananchi_tz_prepaid_standard`, and `yas_sn_fiber_prepayment` (Yas SN PREPAYMENT). 7 programs total. The PREPAID programs follow the Wananchi KE PREPAID example pattern (worked example 5).
3. **Backfill phase** (one-time at V3 cutover): for each existing Subscription that has been escalated to a suspension or similar restricted state in the redeployed codebase, create a corresponding `dunning_state` row at the appropriate level (typically level 3 = SUSPENDED) with `dunning_program_version` pinned to the V3 seed program for that operator. The exact mapping from existing-codebase suspension data → V3 dunning state will be defined during the cartography phase once the existing codebase's data model is known. Active V3 dunning takes over for ongoing escalation.
4. Deploy the Kafka consumers for BIL-01 (`PaymentApplied`), BIL-02 (`InvoiceOverdue`), BIL-03 (`CyclePaymentMissed` covering both PREPAID and PREPAYMENT), and BIL-05 (`WalletToppedUp` for PREPAID auto-recovery) events. Initially, BIL-04 runs alongside whatever existing escalation logic the redeployed codebase has, in a **shadow mode** — observing events, populating state, but not yet invoking workflows. Operators verify state accuracy.
5. Switch over: existing escalation logic is disabled, BIL-04 starts invoking workflows. Frontend admin console becomes the operator surface.
6. The existing escalation logic in the redeployed codebase is retired.

Mapping to TZ baseline (section B) — what V3 adds vs what the redeployed codebase already supports — will be filled in after codebase review.

V3.2 backlog (deferred)

Feature	When it would matter	V3.2 path
Customer-level dunning (across multiple Subscriptions for one customer)	If multi-product customers grow and per-Subscription dunning misses cross-product correlation	Add `customer_dunning_state` aggregating per-Subscription states
Operator-customizable dunning conditions via Drools	If operators need to define VIP-customer exemptions, holiday-pause rules, regional special cases without code changes	Move conditional logic from Java/PHP code to DRL
Soft-dunning notifications before formal level entry	If an operator wants customer-experience-first ops (notify on day 1 of overdue without restricting)	Add `LEVEL_0_SOFT_REMINDER` level type
Predictive scoring (likelihood-of-payment)	If analytics matures to inform escalation decisions	Integrate with CVM/analytics ML output
Dunning case management (CSR-facing workflow tooling)	If recovery escalations need CSR queues and assignment	Build CSR dashboard on top of dunning data